Ingeniería Mecatrónica · Electrónica Digital 2 · Semestre VI

LABORATORIO 3

Lógica secuencial y máquinas de estados finitos

Parte A: divisores de frecuencia y contadores · Parte B: FSM secuenciadora de operaciones

---

Desarrollar en grupos de máximo 3 personas

Profesor: Diego Quiroga · Docente de inicio: Yazmín Bentancour

Entrega: ___ /___ · Reporte de realización del laboratorio (formato IEEE)

Nombre y Apellido: _____	C.I.: _____
Nombre y Apellido: _____	C.I.: _____
Nombre y Apellido: _____	C.I.: _____
Nombre del equipo: _____	Fecha: ___ / ___ / ____

Propósito

Este laboratorio introduce el diseño de circuitos secuenciales (síncronos) en VHDL y culmina con el diseño de máquinas de estados finitos (FSM). En la Parte A el estudiante diseñará un divisor de frecuencia y un contador, implementados de forma que su comportamiento sea directamente observable en los LEDs: a partir del reloj de 100 MHz se generará una señal de 1 Hz que hará parpadear un LED, y un contador que avanzará a la vista. En la Parte B diseñará una FSM que controla una secuencia de operaciones y la integrará con un bloque combinacional que las ejecuta y con el divisor de la Parte A, uniendo así todo lo visto en el curso hasta el momento: lógica combinacional (el cálculo de las operaciones), lógica secuencial (el estado y el reloj) y el concepto de control. No se requiere instrumental adicional (osciloscopio): todo se valida sobre la SEA y el shield.

Materiales

- Spartan Edge Accelerator Board (SEA) — FPGA Xilinx Spartan-7 XC7S15.
- Shield multifunción UTEC (12 dip-switches como entradas y 10 LEDs como salidas).
- Cable USB-C y programador JTAG (HS2).
- Manual de pines de la SEA y del shield UTEC (Moodle).

Software

- Xilinx Vivado (versión 2025.1). Opcional: EDA Playground.

Objetivos del laboratorio

Parte A — Divisores de frecuencia y contadores

- Comprender la diferencia entre lógica combinacional y secuencial, y el papel del reloj.
- Describir un proceso síncrono en VHDL con detección de flanco (rising_edge) y reset.
- Diseñar un divisor de frecuencia mediante un contador, y calcular el factor de división.
- Diseñar un contador de N bits con habilitación (clock enable).
- Usar genéricos para poder simular el diseño sin esperar millones de ciclos de reloj.
- Implementar y validar el sistema en hardware, observándolo en los LEDs.

Parte B — Máquinas de estados finitos

- Comprender el concepto de máquina de estados finitos y sus componentes: estados, transiciones y salidas.
- Distinguir entre máquinas de Moore y de Mealy.
- Describir una FSM en VHDL usando un tipo enumerado para los estados y la plantilla de procesos habitual.
- Integrar la FSM (control) con un bloque combinacional (datapath) y con el divisor de frecuencia de la Parte A.
- Implementar y validar el sistema en hardware.

1. Parte A — Divisores de frecuencia y contadores

A.1 — Conexión con la placa

El reloj de 100 MHz está en el pin H4. El contador de 10 bits se muestra en los 10 LEDs. El reset se toma de un dip-switch (activo en alto), por lo que hay que habilitar el 3.3 V del header Arduino con AR_3V3_EN en '0' (visto en el Laboratorio 1).

Señal	Detalle
clk	Reloj del sistema (100 MHz)
rst	Reset por dip-switch (activo en alto)
LED(0..9)	Contador de 10 bits (LED0 = LSB)
AR_3V3_EN	Habilita el 3.3 V del header Arduino

Recuerde: El uso del dip-switch de reset exige incluir la salida AR_3V3_EN y forzarla a '0' (pin L13). Estándar de E/S: LVCMOS33. El reloj necesita la restricción create_clock en el .xdc.

A.2 — El divisor de frecuencia

Diseñe un módulo divisor que genere un pulso de habilitación (en = '1' durante un ciclo) cada MAX ciclos de reloj. El factor MAX debe ser un genérico. Interfaz sugerida:

```
entity divisor is
    generic ( MAX : natural := 100000000 );  -- ciclos por pulso
    Port ( clk, rst : in  std_logic;
           en       : out std_logic );
end divisor;
```

En la arquitectura, use un contador interno y un proceso síncrono: si el contador llega a MAX−1, reiníciarlo y poner en='1'; en caso contrario, incrementarlo y poner en='0'. Calcule el valor de MAX para obtener 1 Hz desde los 100 MHz.

A.3 — El contador

Diseñe un contador ascendente de N bits con habilitación y reset síncrono, que incremente una vez por cada pulso de en. Interfaz sugerida:

```
entity contador is
    generic ( N : natural := 10 );
    Port ( clk, rst, en : in  std_logic;
           q            : out std_logic_vector(N-1 downto 0) );
end contador;
```

A.4 — Integración

Escriba el top que conecte el reloj con el divisor y el contador, y lleve la salida del contador a los 10 LEDs. Instancie el divisor con el MAX que corresponda a 1 Hz y el contador con N = 10.

```
U_DIV : divisor  generic map ( MAX => 100000000 )
                 port map ( clk => clk, rst => rst, en => en );
```

```
U_CNT : contador generic map ( N => 10 )
                 port map ( clk => clk, rst => rst, en => en, q => LED );
```

A.5 — Simulación

- Escriba un testbench que instancie el divisor (y el contador) con un MAX pequeño (por ejemplo 4), genere el reloj y aplique el reset.
- Verifique que el pulso de enable aparece cada MAX ciclos y que el contador avanza correctamente. Se recomienda un testbench autoverificable (medir el período del enable y comparar el contador con un modelo de referencia).
- Capture las formas de onda para el informe.

A.6 — Implementación en la placa

- Genere el bitstream, programe la placa y observe: el LED0 debe parpadear a 1 Hz y el contador avanzar (un incremento por segundo). Compruebe que el reset pone el contador a cero.
- Grabe un video mostrando el parpadeo y el avance del contador, y el efecto del reset.

Extensión opcional: modifique el genérico para obtener otras frecuencias (por ejemplo, hacer que el contador avance más rápido), o agregue una modulación por ancho de pulso (PWM) para variar el brillo de un LED.

2. Parte B — Máquinas de estados finitos

B.1 — Interfaz y conexión con la placa

La FSM avanza un paso cada vez que recibe un pulso. Ese pulso lo genera el divisor de frecuencia de la Parte A (un tick por segundo), de modo que la secuencia avanza a una velocidad visible. Los operandos A y B se toman de los dip-switches; el resultado y el estado actual se muestran en los LEDs.

Señal	Detalle
clk	Reloj del sistema (100 MHz)
rst	Reset por dip-switch (activo en alto)
A(0..3)	Operando A (4 dip-switches)
B(0..3)	Operando B (4 dip-switches)
LED(3:0)	Resultado de la operación
LED(4)	Carry (en la operación ADD)
LED(9:8)	Operación actual (estado)
AR_3V3_EN	Habilita el 3.3 V del header Arduino

Recuerde: El uso de dip-switches (operandos y reset) exige incluir la salida AR_3V3_EN y forzarla a '0' (pin L13). El reloj necesita la restricción create_clock en el .xdc. Estándar de E/S: LVCMOS33.

B.2 — Especificación del secuenciador

Diseñe una FSM que recorra cíclicamente cuatro estados, cada uno asociado a una operación sobre A y B: S_AND (A and B), S_OR (A or B), S_XOR (A xor B) y S_ADD (A + B). La máquina avanza al siguiente estado con cada pulso del divisor de frecuencia, y el reset la lleva a S_AND. En cada estado se muestra en los LEDs el resultado de la operación y el código de la operación actual.

La salida de Moore op (2 bits) codifica el estado/operación: 00 = AND, 01 = OR, 10 = XOR, 11 = ADD.

B.3 — La máquina de estados

Implemente la FSM en un módulo propio, siguiendo las siguientes indicaciones: tipo enumerado para los estados, proceso síncrono para el registro de estado (reset a S_AND), proceso combinacional para el próximo estado, y la salida de Moore. Complete las cuatro transiciones del case (recuerde que S_ADD vuelve a S_AND).

B.4 — Datapath e integración

Escriba el top que conecte los bloques: el divisor de frecuencia (de la Parte A) genera el pulso; la FSM entrega la operación actual (op); y un bloque combinacional (datapath) aplica esa operación a A y B y lleva el resultado a los LEDs, junto con el código de estado.

B.5 — Simulación

- Escriba un testbench para la FSM que aplique pulsos y verifique la secuencia de estados S_AND → S_OR → S_XOR → S_ADD → S_AND, y el efecto del reset.
- Se recomienda un testbench autoverificable (comparar la salida op contra la secuencia esperada con assert). Verifique también que el datapath entrega la operación correcta.
- Capture las formas de onda para el informe.

B.6 — Implementación en la placa

- Genere el bitstream, programe la placa y observe: el código de operación (LED9:8) avanza cada segundo por la secuencia, y el resultado (LED3:0) cambia según la operación aplicada a A y B. Compruebe el reset.
- Grabe un video mostrando la secuencia de operaciones avanzando y el efecto del reset.

3. Preguntas para el informe

Parte A — Divisores de frecuencia y contadores

- ¿Qué diferencia hay entre un circuito combinacional y uno secuencial? ¿Qué caracteriza a un proceso síncrono en VHDL (rising_edge, reset)?
- Calcule el valor de MAX necesario para obtener 1 Hz a partir de los 100 MHz, y también para 1 kHz. Muestre el cálculo. ¿Cuántos bits necesita el contador interno del divisor para el caso de 1 Hz?
- ¿Por qué se usa un pulso de habilitación (clock enable) para el contador en lugar de usar la señal dividida como reloj del contador? Relaciónelo con las buenas prácticas de diseño en FPGA.
- Explique el uso de genéricos para simular: ¿por qué no conviene simular con el MAX real y cómo lo resolvió en su testbench?
- Observando el contador en los LEDs, ¿a qué frecuencia conmuta cada bit? Relacione cada bit del contador con una división de frecuencia sucesiva del pulso de habilitación.

Parte B — Máquinas de estados finitos

- ¿Qué es una máquina de estados finitos? Describa sus componentes (estados, transiciones, salidas) y explique el diagrama de estados de su diseño.
- ¿Qué diferencia hay entre una máquina de Moore y una de Mealy? ¿De qué tipo es su FSM y por qué?
- ¿Por qué conviene describir los estados con un tipo enumerado en lugar de asignarles códigos de bits a mano? ¿Qué hace la herramienta de síntesis con esa codificación?
- Explique la plantilla de procesos que usó. ¿Por qué el registro de estado va en un proceso síncrono y la lógica de próximo estado en uno combinacional?
- Este laboratorio integra una FSM (control) con un datapath (operaciones). Relacione esta estructura con la de un procesador: ¿qué papel cumple cada parte?

4. Entregables

El desarrollo debe ser original, por grupos de un máximo de 3 personas. Los trabajos idénticos reportados por separado serán causal de anulación de todos los grupos involucrados. Deberá entregar:

- Código fuente VHDL completo de la Parte A (divisor, contador, top y testbench).
- Código fuente VHDL completo de la Parte B (FSM, datapath, top y testbench).
- Resultados de la simulación de ambas partes (capturas de las formas de onda).
- Un video de la simulación y el funcionamiento en la placa por cada parte: en la Parte A, el parpadeo del LED0, el avance del contador y el reset; en la Parte B, la secuencia de operaciones y el reset.
- Un único informe en formato IEEE que abarque ambas partes y responda las preguntas de la sección 3, incluyendo el diagrama de estados de la Parte B.

5. Rúbrica de evaluación

La evaluación utiliza la rúbrica por niveles de logro de la asignatura (Destacado / Competente / En desarrollo / Inicial); consulte el documento de rúbrica para los descriptores de cada nivel. Los criterios y sus pesos son: Cada criterio se evalúa considerando ambas partes del laboratorio.

Criterio	Peso
Funcionamiento en la placa	30 %
Simulación (testbench)	20 %
Video de testing (simulación e implementación)	10 %
Informe: claridad y organización	15 %
Informe: preguntas específicas	15 %
Informe: conclusiones	10 %

ES OBLIGATORIO PRESENTAR EL FUNCIONAMIENTO DEL CIRCUITO DE FORMA PRESENCIAL, coordinando previamente con los docentes.